Perimity — Software Requirements Specification (SRS) v1.1

Smart Campus Access & Gate Pass Management System CDAC Mumbai · 6-person team · Microservices architecture

> Companion docs (read alongside this one): `Perimity_Database_Design.md`, `Perimity_Event_Bulk_Design.md`. This SRS adds functional requirements, per-service API contracts, and the frontend-ownership model on top of what those two docs already define.

---

1. Introduction

1.1 Purpose

This document tells each team member exactly what their service must do — its data, its API, its screens, and its testing method — so six people can build in parallel without stepping on each other.

1.2 Scope

Perimity replaces the paper gate register with a digital, forgery-proof, searchable system. Entry-only. Passwordless (email + OTP) login for every user type. Supports both permanent (student/faculty) passes and time-boxed event passes, bulk onboarding via Excel, and QR-based gate scanning with instant GREEN/RED/AMBER results.

1.3 Glossary

Term	Meaning
Identity	Who someone is — one per person, forever, keyed by email
Pass	Permission to enter, for a purpose, for a time window. One identity can hold several passes at once
DAILY pass	Standing pass for students/faculty, no end date
EVENT pass	Time-boxed pass tied to a specific event's date range
OTP	One-time password, emailed, used instead of a real password

Term	Meaning
Guard scan	A QR scan at a gate; always returns GREEN, RED, or AMBER
DAC / DBDA / DESD / DITISS / DMLT / PGAIML	CDAC course codes used as “departments” — there is no semester concept

1.4 References

- `Perimity_Database_Design.md` — schema, database-per-service map, S3 rules
- `Perimity_Event_Bulk_Design.md` — identity vs pass model, bulk engine, gate scan logic

---

2. Overall Description

2.1 Product perspective

Six independent Spring Boot services, each with its own database, each exposing a REST API documented via Swagger. A React frontend calls these APIs directly. RabbitMQ handles anything slow/async (QR + PDF generation, bulk-upload processing, emails). No service reads another service’s database — only its API.

2.2 User classes

User	Can do
Visitor	Register for a visit, verify via OTP, view/download their pass
Event visitor	Same as visitor, pass scoped to one event
Student / Faculty	Login via OTP, hold a DAILY pass, optionally also an EVENT pass
Faculty (bulk uploader)	Upload Excel batches (students or event visitors), monitor processing
Admin	Manage campuses, gates, config, departments, view audit logs
Guard	Scan QR codes at a gate, see GREEN/RED/AMBER result
Event organizer	View live attendance dashboard for their event

2.3 Operating environment

Docker Compose locally (all 6 services + Postgres + MongoDB + RabbitMQ + Redis). AWS (EC2 + S3) for real deployment, later.

2.4 Design constraints (apply to every service, no exceptions)

- **No `Semester` field in any UI**, even if a schema has one.
- Login is passwordless everywhere — email + OTP only.
- Files (photos, Aadhaar, QR images, PDFs) live in S3; the database stores only the S3 key.
- Passwords: bcrypt. OTPs: SHA-256 hashed. QR tokens: AES-256 encrypted.
- Every service must expose Swagger UI (see Section 7) — this is how teammates test and integrate with each other's APIs before any frontend exists.
- No service calls another service's database directly.

---

3. System Architecture Overview

Service	DB	Type	Port	Owner builds
Auth Service	AuthDB	PostgreSQL	5432	backend + login/OTP screens
User Service	UserDB	PostgreSQL	5433	backend + profile screens
Gate Pass Service	GatePassDB	PostgreSQL	5434	backend + visitor/pass/bulk screens
Campus Service	CampusDB	PostgreSQL	5435	backend + admin campus/gate screens
Guard Service	EntryLogDB	MongoDB	27017	backend + guard scanner screen
QR Service	QRDB	PostgreSQL	5436	backend + pass download screen

Ownership model (important — read Section 6 fully): there is no separate "frontend person." Whoever owns a service owns its backend API **and** the React pages/components that call it.

---

4. Functional Requirements by Service

Each section below is self-contained — a teammate only needs their own section, this document's Sections 1–2 and 6–7, plus the two companion design docs, to start building.

4.1 Auth Service — AuthDB (PostgreSQL, port 5432)

Entities: `users`, `otp_verifications`, `audit_logs` (see Database Design doc for columns).

API endpoints:

Method	Path	Purpose
POST	`/api/auth/otp/request`	Body `{email}` → generates OTP, emails it, stores SHA-256 hash
POST	`/api/auth/otp/verify`	Body `{email, otp}` → validates, returns session token (JWT)
GET	`/api/auth/me`	Returns current logged-in user's basic info
POST	`/api/auth/logout`	Invalidates session
GET	`/api/auth/audit-logs`	Admin-only, paginated audit trail
POST	`/api/auth/users`	Internal — called by User Service when a new identity is created

Frontend screens this owner builds:

- Email entry screen ("enter your email to continue")
- OTP entry screen (6-digit code, resend button, 10-min countdown)
- This is shared UI every other service's pages will redirect to when a user isn't logged in — build it as a standalone reusable flow.

Business rules: OTP expires in 10 minutes, locks after 5 wrong attempts, never stored or logged in plain text.

4.2 User Service — UserDB (PostgreSQL, port 5433)

Entities: `student_profiles`, `faculty_profiles`, `departments`, `documents`.

API endpoints:

Method	Path	Purpose
GET	`/api/users/students/{id}`	Fetch one student profile
POST	`/api/users/students`	Create a student profile (calls Auth Service to create the identity first)
GET	`/api/users/faculty/{id}`	Fetch one faculty profile
POST	`/api/users/faculty`	Create a faculty profile
GET	`/api/users/departments`	List departments (DAC, DBDA, DESD, DITISS, DMLT, PGAIML)
POST	`/api/users/documents`	Register an uploaded document's S3 key + metadata

Frontend screens this owner builds:

- Student profile view/edit page (no semester field!)
- Faculty profile view/edit page
- Department picker component (reused wherever a department dropdown is needed)
- Document upload widget (uploads to S3 via a pre-signed URL, then calls the API above with the resulting key)

4.3 Gate Pass Service — GatePassDB (PostgreSQL, port 5434)

Entities: `visitor_requests`, `gate_passes` (with `pass_type`, `event_id`), `bulk_upload_batches`, `events`.

API endpoints:

Method	Path	Purpose
POST	`/api/gatepass/visitor-requests`	Visitor submits the registration form
PUT	`/api/gatepass/visitor-requests/{id}/approve`	Admin/faculty approves → triggers pass creation
GET	`/api/gatepass/passes/me`	Logged-in user's own active pass(es)
GET	`/api/gatepass/passes/{id}`	Fetch one pass
POST	`/api/gatepass/events`	Create an event (name, campus, date range)
GET	`/api/gatepass/events/{id}/attendance`	Live attendance view: registered / attended per day / never showed
POST	`/api/gatepass/bulk-upload`	Upload Excel (students or event visitors)
GET	`/api/gatepass/bulk-upload/{batchId}/status`	Poll validation/processing status

Frontend screens this owner builds:

- Visitor registration form
- Faculty/admin bulk-upload page: file picker → "580 valid, 20 errors" summary → confirm button → "done, passes generating" message
- Pass view page (shows QR + validity dates; may hold both a DAILY and an EVENT pass)
- Organizer attendance dashboard (matches the mockup in the Event & Bulk Design doc: Registered / Attended Day 1 / Attended Day 2 / Never showed / export CSV)

Business rules: one bulk engine for students and event visitors — only the `type` column and date source differ. Mixed batches resolved per-row by email (existing identity reused, new email gets a lightweight visitor identity). See Event & Bulk Design doc for the full flow.

4.4 Campus Service — CampusDB (PostgreSQL, port 5435)

Entities: `campuses`, `campus_gates`, `campus_config`.

API endpoints:

Method	Path	Purpose
GET	`/api/campus`	List all campuses
POST	`/api/campus`	Create a campus (admin only)
GET	`/api/campus/{id}/gates`	List gates for a campus
POST	`/api/campus/{id}/gates`	Add a gate
GET	`/api/campus/{id}/config`	Get campus settings (approval required? re-entry allowed?)
PUT	`/api/campus/{id}/config`	Update campus settings

Frontend screens this owner builds:

- Admin: campus list + create/edit page
- Admin: gate management page per campus
- Admin: campus config/settings page

4.5 Guard Service — EntryLogDB (MongoDB, port 27017)

Entities: `entry_logs` (one document per scan).

API endpoints:

Method	Path	Purpose
POST	`/api/guard/scan`	Body `{token, gateId, guardId}` → decrypts token (calls QR Service), returns `{result: GREEN|RED|AMBER, message}`
GET	`/api/guard/logs`	Query by campus + date range

Method	Path	Purpose
GET	`/api/guard/logs/pass/{passId}`	All scan history for one pass

Frontend screens this owner builds:

- Guard scanner screen: camera/QR input, then a full-screen colored result card (green/red/amber) with the message (“Welcome”, “Welcome to AI Summit”, or the denial reason)
- Guard’s own scan history list

Business rules: entry-only, no exit scan. If the pass is a DAILY pass but the person has an event running today, auto-attribute the entry to that event (Behavior 2 in the Event & Bulk Design doc) — this logic lives here, not in the frontend.

4.6 QR Service — QRDB (PostgreSQL, port 5436)

Entities: `qr_records`, `generation_jobs`.

API endpoints:

Method	Path	Purpose
GET	`/api/qr/{passId}`	Get QR/PDF S3 keys + validity for a pass
GET	`/api/qr/{passId}/download`	Redirect/stream the PDF
GET	`/api/qr/jobs/{jobId}/status`	Check status of an async generation job
POST	`/api/qr/decrypt`	Internal — called by Guard Service to decrypt a scanned token

Frontend screens this owner builds:

- Pass download page (view QR image, download PDF button)
- Bulk-generation progress indicator (for admin/faculty watching hundreds of jobs process)

Business rules: consumes jobs from RabbitMQ (dropped by Gate Pass Service), generates a signed AES-256 token, a QR PNG, and a PDF, uploads both to S3, then updates the job + notifies Gate Pass Service to flip the pass to Active.

---

5. Non-Functional Requirements

- **API docs:** every service must expose Swagger — see Section 7.

- **Security:** no plaintext secrets anywhere, ever — see Section 2.4.
- **Async by default:** anything involving more than ~10 QR/PDF generations or emails must go through RabbitMQ, never be done synchronously in a request.
- **Consistency:** REST paths follow `/api/<service-noun>/<resource>` exactly as listed above, so other teammates' frontends can call them predictably.

---

6. Frontend Ownership Model

There is **no dedicated frontend-only teammate**. Each of the 6 members owns a full vertical slice:

1. Their Spring Boot service (backend + database + Swagger docs)
2. The React pages/components that call *only* their own service's API

Why this works: nobody has to wait on anyone else to see their own feature working end-to-end — you can build and test your backend via Swagger first, then wire up your own React pages against it.

Repo layout for the frontend:

```
frontend/
├── src/
│   ├── auth/          ← Auth Service owner's pages
│   ├── users/         ← User Service owner's pages
│   ├── gatepass/       ← Gate Pass Service owner's pages
│   ├── campus/         ← Campus Service owner's pages
│   ├── guard/          ← Guard Service owner's pages
│   ├── qr/             ← QR Service owner's pages
│   └── shared/         ← shared layout, routing, auth context (small, touched via PR by whoever needs
```

Each person adds their own folder under `src/` and only touches `shared/` when genuinely necessary (via PR, not directly).

During development, each person's React pages call their own backend directly at its own port (e.g. `http://localhost:8081` for Auth). No API gateway yet — that can be added later if needed.

---

7. API Documentation — Swagger (required for every service)

Every Spring Boot service must include this Maven dependency in its `pom.xml`:

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

```
    <version>2.6.0</version>
</dependency>
```

No extra configuration is required — once this dependency is added and the service starts, Swagger UI is automatically available at:

```
http://localhost:<service-port>/swagger-ui.html
```

and the raw OpenAPI spec at:

```
http://localhost:<service-port>/v3/api-docs
```

Why this matters for the team: Swagger UI lets every teammate test and call any service's API directly in the browser — with a working "Try it out" button — before any frontend page exists for it. This is how you'll integrate with each other's services without waiting for anyone's React pages to be done.

Rule: every `@RestController` method should have a short `@Operation(summary = "...")` annotation so the Swagger page is readable, not just a list of raw paths.

---

8. Appendix — Where to find the rest

- Full database schema, table columns, S3 folder structure → `Perimity_Database_Design.md`
- Identity/pass model, bulk engine, mixed-attendee resolution, full gate scan decision tree, organizer dashboard mockup → `Perimity_Event_Bulk_Design.md`
- Repo setup, branching, CI → root `README.md` and `.github/workflows/docker-build.yml`